

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



ACTIVIDAD PRÁCTICA

ANÁLISIS DISTRIBUIDO DE DATOS METEOROLÓGICOS MEDIANTE MPI

ASIGNATURA

SISTEMAS DISTRIBUIDOS

GRUPO 2

DOCENTE

MOLINA BARRIGA, MARIBEL

INTEGRANTES

BEDREGAL PEREZ, DANIEL

JARA MAMANI, MARIEL ALISSON

MESTAS ZEGARRA, CHRISTIAN RAUL

NOA CAMINO, YENARO JOEL

SEQUEIROS CONDORI, LUIS GUSTAVO

AREQUIPA - PERÚ

2026

ANÁLISIS DISTRIBUIDO DE DATOS METEOROLÓGICOS MEDIANTE MPI

1. Enlace a GitHub

<https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/teo/mpi>

2. Introducción

El procesamiento de grandes volúmenes de datos meteorológicos requiere infraestructuras capaces de manejar cargas computacionales intensivas de manera eficiente. La computación distribuida surge como una solución robusta, permitiendo el reparto de tareas entre múltiples nodos de procesamiento. En este proyecto, se implementa un sistema de análisis meteorológico utilizando el estándar MPI (Message Passing Interface) a través de la librería `mpi4py`. El enfoque principal reside en la orquestación de un cluster virtualizado y la eficiencia de la comunicación inter-proceso para el procesamiento paralelo de datos masivos.

3. Marco Teórico

La computación distribuida se basa en el uso de múltiples sistemas autónomos que se comunican a través de una red para alcanzar un objetivo común. Los modelos de programación paralela han evolucionado para abordar diferentes jerarquías de hardware (Czarnul et al., 2020).

MPI es el estándar de facto para la comunicación en sistemas de memoria distribuida, permitiendo la transferencia de datos entre procesos independientes mediante el paso explícito de mensajes (Dalcin et al., 2011). A diferencia de los modelos de memoria compartida, MPI requiere que el programador gestione manualmente la distribución de los datos y la sincronización, lo que ofrece un control granular y una escalabilidad casi lineal en infraestructuras de gran tamaño (Ashraf et al., 2016).

El modelo Maestro-Trabajador es un patrón de diseño donde un nodo central coordina la distribución de datos y la recolección de resultados, mientras que los nodos esclavos ejecutan las tareas computacionales. En el contexto de Python, `mpi4py` proporciona una interfaz idiomática que permite aprovechar las capacidades de MPI integrándose con el ecosistema de ciencia de datos (Pandas, Scikit-learn).

4. Arquitectura Propuesta

La arquitectura se basa en una red de contenedores Docker que emulan un cluster de alto rendimiento (HPC). Esta configuración permite simular nodos físicos independientes con entornos de ejecución aislados, como se detalla en la Figura 1.

4.1. Infraestructura de Contenedores

El cluster se despliega mediante Docker Compose, definiendo tres tipos de servicios: `master`, `worker` (escalable) y `dashboard`. La robustez de la arquitectura reside en los siguientes pilares técnicos:

- 1. Comunicación entre nodos vía SSH:** Cada contenedor ejecuta un servidor OpenSSH, así `mpirun` puede lanzar procesos de forma remota entre los contenedores de la red interna.

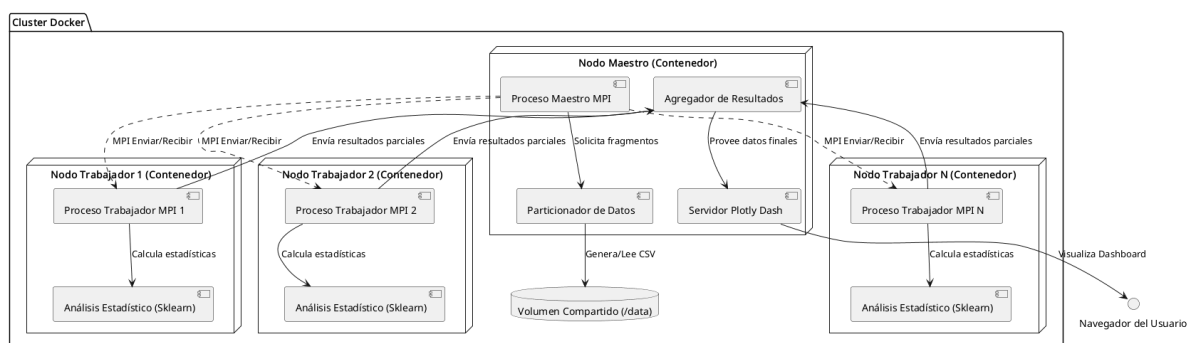


Figura 1. Diagrama de Arquitectura

2. **Descubrimiento Dinámico:** El nodo maestro utiliza herramientas de red como dig para identificar las direcciones IP de los trabajadores a través del DNS interno de Docker.
3. **Almacenamiento:**
 - **Volúmenes Locales:** Se utilizan para persistir el dataset de entrada.
 - **Memoria Compartida (tmpfs):** Se monta un volumen de alto rendimiento basado en RAM (shm_data) para el intercambio rápido de resultados JSON y archivos temporales, minimizando la latencia de I/O de disco durante la agregación final.

4.2. Operaciones Colectivas MPI

La comunicación se hace mediante el uso de operaciones colectivas:

- **Scatter:** Distribuye equitativamente fragmentos del dataset global desde la memoria del maestro hacia los trabajadores.
- **Gather:** Recolecta y consolida los objetos de resultados calculados de forma independiente.

5. Desarrollo de la Solución

La solución integra la generación de datos, el procesamiento distribuido y la visualización final.

5.1. Generación de Datos

Se utiliza un modelo basado en funciones sinusoidales para generar un millón de registros que incluyen variables de temperatura, humedad y viento, incorporando estacionalidad y sesgos geográficos para simular condiciones meteorológicas reales.

5.2. Procesamiento y Modelado

Cada proceso trabajador recibe un fragmento de datos y realiza un análisis estadístico exhaustivo. Además, se emplea un modelo de regresión lineal de Scikit-learn para proyectar tendencias futuras a partir de los datos procesados localmente. El enfoque aquí no es la precisión del modelo, sino la capacidad del sistema para distribuir la carga de entrenamiento y predicción entre múltiples nodos.

5.3. Visualización

Los resultados agregados se visualizan en un dashboard interactivo desarrollado en Plotly Dash, que consume los datos procesados almacenados en el volumen de memoria compartida para garantizar una respuesta fluida de la interfaz (ver Figura 2 y Figura 3)



Figura 2. Dashboard: Sección superior

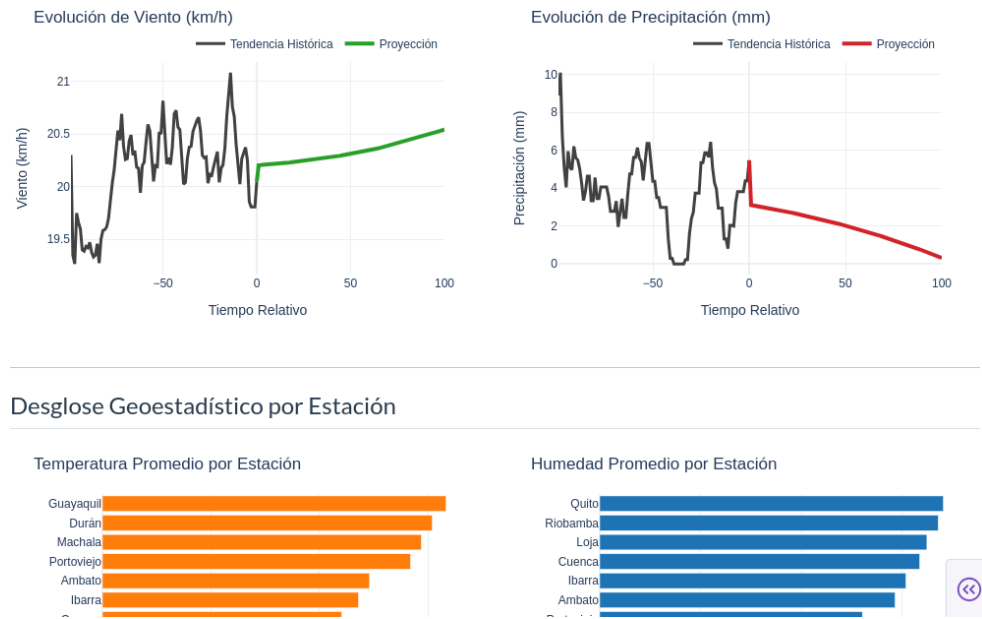


Figura 3. Dashboard: Sección inferior

6. Resultados

Se realizaron pruebas de rendimiento comparando una ejecución secuencial frente a una distribuida con 4 trabajadores (ver Figura 4).

Estrategia	Ejecución (s)
Secuencial	8.841
MPI	11.166

Aunque el tiempo MPI es superior en este entorno virtualizado, este comportamiento se justifica por dos factores técnicos críticos:

- Docker y Red Virtual:** El uso de contenedores en una sola máquina física introduce una latencia de red virtual y sobrecarga de gestión de recursos que no existe en un entorno bare-metal (Krzywaniak et al., 2022). En un entorno de producción HPC, la interconexión de baja latencia (como InfiniBand) permitiría que el paralelismo supere rápidamente los retrasos en comunicación.
- Eficiencia de Scikit-learn:** Las rutinas internas de Scikit-learn ya están altamente optimizadas mediante OpenMP y BLAS, lo que permite que la ejecución secuencial aproveche varios núcleos de manera eficiente sin la sobrecarga de paso de mensajes de MPI (developers, 2024).

No obstante, la arquitectura demuestra su capacidad para manejar volúmenes de datos que excederían la memoria de un solo nodo, justificando su uso por la escalabilidad horizontal y la resiliencia en el manejo de grandes conjuntos de datos.

```
Generating data...
Generated 1000000 rows -> /data/input.csv
Generated 1000000 rows -> /data/input.csv
Benchmark...
Benchmark 1: uv run scripts/run_sequential.py
Time (mean ± σ):      8.841 s ± 0.168 s    [User: 14.064 s, System: 0.846 s]
Range (min .. max):   8.747 s .. 8.946 s    3 runs

Benchmark 2: bash scripts/run_mpi.sh 4
Time (mean ± σ):      11.166 s ± 0.059 s   [User: 39.542 s, System: 14.024 s]
Range (min .. max):   11.099 s .. 11.214 s   3 runs

Summary
'uv run scripts/run_sequential.py' ran
1.26 ± 0.02 times faster than 'bash scripts/run_mpi.sh 4'
```

Figura 4. Resultados del Benchmark con Hyperfine

7. Discusión

La elección de MPI frente a otras tecnologías se fundamenta en sus características específicas para HPC y sistemas distribuidos:

- **MPI vs OpenMP:** OpenMP es excelente para paralelismo en memoria compartida, pero está limitado a una única máquina física. MPI, aunque conlleva una mayor complejidad por el paso de mensajes explícito, permite escalar a miles de nodos interconectados (NVIDIA, 2017).
- **MPI vs CUDA:** CUDA permite una aceleración masiva en GPUs para tareas masivamente paralelas (SIMD). Sin embargo, requiere hardware propietario y sufre de cuellos de botella en la transferencia de datos entre CPU y GPU. MPI es agnóstico al hardware y es preferible para tareas que requieren gran capacidad de memoria distribuida (Ashraf et al., 2016).
- **MPI vs Ray:** Ray es un framework moderno diseñado para aplicaciones de IA con escalado dinámico. Aunque es más flexible y maneja fallos de forma automática, MPI ofrece un rendimiento superior y menor latencia en aplicaciones científicas con patrones de comunicación estáticos y predecibles (Dagher, 2023).

8. Conclusiones

El sistema implementado demuestra que la combinación de MPI y Docker proporciona una plataforma robusta para el procesamiento distribuido. La arquitectura de comunicación basada en SSH y el uso de volúmenes en memoria optimizan el rendimiento en entornos virtualizados. Aunque la virtualización introduce latencias, la capacidad de escalabilidad horizontal posiciona a esta solución como una base sólida para aplicaciones de procesamiento de datos meteorológicos a gran escala en infraestructuras de nube.

9. Referencias

- Ashraf, M. U., Fouz, F., & Eassa, F. A. (2016). Empirical analysis of HPC using different programming models. *International Journal of Modern Education and Computer Science*, 8(6), 27-34. <https://doi.org/10.5815/ijmecs.2016.06.04>
- Czarnul, P., Proficz, J., & Drypczewski, K. (2020). Survey of Methodologies, Approaches, and Challenges in Parallel Programming Using High-Performance Computing Systems. *Scientific Programming*, 2020. <https://doi.org/10.1155/2020/4176794>
- Dagher, P. (2023). *Ray vs Spark — The Future of Distributed Computing*. <https://medium.com/@nasdag/ray-vs-spark-the-future-of-distributed-computing-b10b9caa5b82>
- Dalcin, L. D., Paz, R. R., Kler, P. A., & Cosimo, A. (2011). Parallel Python with MPI for Python. *Advances in Water Resources*, 34(9), 1124-1139. <https://doi.org/10.1016/j.advwatres.2011.04.013>
- developers. (2024). *Parallelism, resource management, and configuration*. <https://scikit-learn.org/stable/computing/parallelism.html>
- Krzywaniak, A., Proficz, J., & Czarnul, P. (2022). Performance Assessment of Using Docker for Selected MPI Applications in a Parallel Environment Based on Commodity Hardware. *Applied Sciences*, 12(16), 8305. <https://doi.org/10.3390/app12168305>
- NVIDIA. (2017,). *When to use Serial CPU, CUDA, OpenMP and MPI?*. <https://forums.developer.nvidia.com/t/when-to-use-serial-cpu-cuda-openmp-and-mpi/48379>